

EURO - INSIGHT

Raúl Jiménez — `raul.jimenez8@alu.uclm.es`

Sistema de recomendación contextual de compra de Euromillones
Descripción de producto Big Data para la asignatura
(16362) PROCESAMIENTO MASIVO DE DATOS

Índice

1. Problema al que pretende atacar	3
2. Preguntas que ayuda a responder	4
2.1. Patrones numéricos	4
2.2. Patrones temporales	4
2.3. Patrones territoriales	4
3. Conjuntos de datos y velocidad	5
3.1. Datos Estructurados	5
3.1.1. Batch (estructurado)	5
3.1.2. Optimizaciones del pipeline de ingesta CSV	5
3.2. Streaming	6
3.2.1. Configuración de watermark, trigger y agregaciones	6
3.3. Almacenamiento estructurado (Base de Datos Relacional)	7
3.3.1. Estrategia de ingesta incremental	7
4. Arquitectura Medallion y Delta Lake	9
4.1. Capa Bronze — datos en crudo	9
4.2. Capa Silver — datos validados y normalizados	9
4.3. Capa Gold — agregaciones para consumo	10
4.4. Programación temporal e integración batch-streaming	10
5. Datos necesarios	12
5.1. Datos históricos Euromillones (Batch)	12
5.2. Datos demográficos provinciales (Batch - INE)	12
5.3. Datos de puntos de venta (Batch - SELAE CSV)	13
5.4. Datos de streaming en tiempo real (Kafka)	13
5.5. Datos post-sorteo (Streaming - SELAE scraping)	13
6. Arquitectura técnica y flujo de datos	15
6.1. Stack tecnológico	15
6.2. Flujo de datos	15
6.2.1. Pipeline batch (ejecución diaria):	15
6.2.2. Pipeline streaming (ejecución continua):	15
6.2.3. Post-sorteo (martes y viernes):	15

Alcance y reglas

Euromillones es el mismo sorteo europeo desde 2004, con reglas estables: 5 números (1–50) + 2 estrellas (1–12). Existen datasets históricos en CSV y una fuente oficial en España para publicar resultados tras cada sorteo. Datos demográficos o económicos son recogidos, como regla general, en el Instituto Nacional de Estadística (INE).

- Kaggle – Dos datasets:
 - Euromillones Draw Numbers (2004–2023): CSV con fecha, números, estrellas y recaudación en España (útil para enriquecer). *Autor: Miguel Minambres*.
 - Lottery EuroMillions Data (2004-2024): CSV del históricos de Reino Unido desde 2004, por lo que incluye su moneda, la libra esterlina. *Autor: Sihan Tao*. Se utilizará como fuente secundaria para:
 - (a) validación cruzada de números ganadores por fecha
 - (b) backfill de sorteos faltantes en el dataset español
 - (c) análisis comparativo de patrones culturales entre España y UK
- Página oficial de INE: útil para datos demográficos/económicos.
- Página oficial de SELAE – Resultados Euromillones: actualiza tras cada sorteo (útil para scraping).

1. Problema al que pretende atacar

Crear un sistema que detecte patrones temporales, numéricos y territoriales en los resultados de **Euro-millones** y ayude al usuario a:

- Detectar tendencias en los números (calientes/fríos/volatilidad).
- Observar si existen patrones estacionales (por meses, días de la semana, épocas del año).
- Identificar localidades/administraciones que históricamente reparten más premios (con las limitaciones actuales → no existe *dataset* abierto perfecto en España, así que lo reemplazamos con una proxy válida: SELAE).
- Enriquecer el análisis con datos demográficos y socioeconómicos (población, renta media, tasa de desempleo) para identificar perfiles provinciales con mayor propensión al juego.
- Generar una estimación probabilística contextual (no adivinación) para decidir → qué combinación podría ser más interesante jugar según historial + contexto actual.

Importante: el sistema **NO predice números ganadores**, sino que ofrece un análisis probabilístico contextual basado en patrones reales.

El punto fuerte de emplear el sorteo de Euromillones es que abre la puerta a usar conjuntos de datos de diferentes regiones a nivel europeo, puesto que **es un único sorteo europeo**; siempre con las mismas reglas y múltiples fuentes compatibles. Adicionalmente, el sistema incorporará datos estructurados de fuentes oficiales (INE, SELAE, Open Data España) para enriquecer el análisis con variables demográficas, económicas y territoriales, permitiendo identificar correlaciones entre características provinciales y comportamiento de juego.

Variedad: dos fuentes batch distintas (Kaggle primario + CSV alternativo), con columnas y formatos que obligan a normalizar, aumentando robustez. En definitiva, se tratan de **múltiples orígenes del mismo sorteo**; no se tiende a mezclar loterías incompatibles.

2. Preguntas que ayuda a responder

Podemos distribuir preguntas propuestas en función de los patrones:

2.1. Patrones numéricos

- ¿Qué números aparecen más frecuentemente en los últimos X sorteos?
- ¿Hay combinaciones frecuentes de pares/impares?

2.2. Patrones temporales

- ¿Hay meses con mayor aparición de ciertos números?
- ¿Hay diferencias entre los sorteos del martes y del viernes?

2.3. Patrones territoriales

- ¿Qué provincias tienen mayor densidad de administraciones por cada 10.000 habitantes?
- ¿Existe correlación entre la cercanía de una administración y el gasto provincial en Euromillones?
- ¿Las administraciones en zonas turísticas (costa, capitales) venden más que las rurales?
- En Madrid, ¿cuál es la distancia media entre administraciones y cómo afecta a la competencia?
- ¿Hay clusters geográficos de administraciones "afortunadas" (que han repartido múltiples premios)?

3. Conjuntos de datos y velocidad

3.1. Datos Estructurados

- **INE (Instituto Nacional de Estadística)**: Datos demográficos y socioeconómicos por provincia descargables en CSV, los cuales *se usarán para un modelo de datos estructurado*:

- Población y densidad por provincia/municipio
- Renta media por hogar
- Tasa de desempleo provincial
- PIB per cápita

Ficheros descargables en **diversos formatos** (Excel, CSV, CSV (;), Pc-Axis, JSON) Actualización: **anual**. <https://www.ine.es/dyngs/INEbase/listaoperaciones.htm>
Datos a consultar involucrados:

- Cifras oficiales de población de los municipios españoles en aplicación de la Ley de Bases del Régimen Local (Art. 17)
- Renta por hogar por comunidades autónomas

- **SELAE**: CSV oficial de puntos de venta de lotería. Dataset estructurado con:

- Longitud (*float*)
- Latitud (*float*)
- Dirección del punto de venta (*string*)

Descarga directa en formato CSV, actualización **semestral**.

https://www.loteriasypuestas.es/f/loterias/documentos/geo/poi_csv.zip

Requerirá un proceso de transformación para que encaje con la estructura de datos para la BD.

3.1.1. Batch (estructurado)

- Kaggle – Euromillones Draw Numbers (principal): CSV completo, incluye recaudación en España. <https://www.kaggle.com/datasets/miguelminambres/euromillones-draw-numbers?select=Euromillones+Draw+Data.csv>
- Kaggle – Lottery EuroMillions Data como CSV alternativo (válido para robustez y *backfill*): descarga CSV completo desde 2004. <https://www.kaggle.com/datasets/sihantao0626/lottery-euromillions-dataset>

3.1.2. Optimizaciones del pipeline de ingesta CSV

El pipeline de ingesta de ficheros CSV incorpora las siguientes optimizaciones, aplicadas en fases distintas del proceso:

Particionado por dimensión temporal. Los datos se escriben en las capas Bronze y Silver de Delta particionados por **año** y **mes**. Esto permite a Spark aplicar *partition pruning* en consultas analíticas acotadas temporalmente, evitando la lectura de particiones irrelevantes y reduciendo significativamente el volumen de datos deserializados.

Schema enforcement en la transición Bronze → Silver. La capa Silver define un schema estricto sobre el que Delta aplica validación en escritura. Los registros que no lo satisfagan (fechas malformadas frecuentes en el CSV de Reino Unido, valores nulos en campos obligatorios como **numero_1**) son derivados a una tabla de cuarentena **sorteos_quarantine** para su revisión, sin interrumpir el pipeline principal.

Pushdown de predicados. Cuando el caso de uso requiere únicamente un subconjunto temporal reciente, el pipeline aplica filtros tempranos sobre `fecha_sorteo` antes de cualquier transformación. Combinado con el particionado descrito anteriormente, Spark traduce dichos filtros directamente a rutas de ficheros Delta, sin deserializar particiones fuera del rango de interés.

Mapa de normalización de columnas. Dado que los dos datasets CSV de Kaggle presentan esquemas heterogéneos (nombres de columna distintos, separadores y encodings diferentes), el pipeline Silver mantiene un mapa de normalización explícito que garantiza la estabilidad del schema unificado ante futuras versiones de los datasets fuente. La Tabla 1 recoge la correspondencia de columnas principales entre ambas fuentes.

Campo unificado	Dataset ES (Minambres)	Dataset UK (Tao)
<code>fecha_sorteo</code>	<code>Date</code>	<code>DrawDate</code>
<code>numero_1</code>	<code>N1</code>	<code>Ball 1</code>
<code>estrella_1</code>	<code>LS1</code>	<code>Lucky Star 1</code>
<code>recaudacion</code>	<code>Recaudacion.Espana</code>	<code>n/a</code>

Cuadro 1: Mapa de normalización de columnas entre datasets CSV.

3.2. Streaming

- **Generador sintético de transacciones de apuesta:** Simulador de eventos de compra de boletos en tiempo real basado en patrones históricos. Genera eventos cada 100-500ms con:

- timestamp, provincia, administración, números apostados
- importe, tipo de apuesta (simple/múltiple)
- canal (presencial/online)

Publicación a Apache Kafka (topic: `apuestas-raw`)

- Pipeline de procesamiento continuo: Apache Spark Structured Streaming consume eventos de Kafka y realiza:
 - Agregaciones por provincia en ventanas temporales (1 minuto)
 - Enriquecimiento con datos batch (PostgreSQL): población, históricos
 - Cálculo de métricas en tiempo real: gasto acumulado 48h, números más apostados, índice de probabilidad contextual
 - Publicación a topics de salida: `probabilidad-provincial`, `tendencias-numeros`

3.2.1. Configuración de watermark, trigger y agregaciones

El pipeline de Spark Structured Streaming procesa eventos en *event-time*, tomando el campo `timestamp` del mensaje Kafka como referencia temporal. Este enfoque permite gestionar correctamente eventos que lleguen con retraso respecto al momento real de la apuesta.

Watermark. Se define un watermark de **2 minutos** sobre el campo `timestamp`:

```
.withWatermark("timestamp", "2 minutes")
```

Spark descartará los eventos cuyo `timestamp` sea más de 2 minutos anterior al máximo timestamp observado, liberando el estado acumulado de ventanas cerradas y acotando el uso de memoria del executor.

Trigger. El pipeline utiliza un trigger de *processing time* cada 30 segundos:

```
.trigger(Trigger.ProcessingTime("30 seconds"))
```

Este intervalo equilibra la frescura del dato —suficiente para actualizar el índice de probabilidad contextual provincial— con el coste computacional del join continuo contra las tablas Delta batch.

Ventanas de agregación. Se emplean *tumbling windows* de 1 minuto agrupadas por `codigo_provincia`:

```
df.withWatermark("timestamp", "2 minutes")
  .groupBy(
    window(col("timestamp"), "1 minute"),
    col("codigo_provincia")
  )
  .agg(
    sum("importe").alias("gasto_acumulado"),
    count("id_transaccion").alias("num_transacciones"),
    collect_list("numeros").alias("numeros_apostados"),
    first("canal").alias("canal_predominante")
  )
```

Modo de escritura. Los resultados se escriben en la capa Gold de Delta en modo **append**, dado que el watermark garantiza que las ventanas se emitan únicamente cuando están cerradas, sin actualizaciones retroactivas:

```
.writeStream
  .format("delta")
  .outputMode("append")
  .trigger(Trigger.ProcessingTime("30 seconds"))
  .option("checkpointLocation", "/delta/gold/checkpoints/apuestas")
  .toTable("gold.agregaciones_provinciales")
```

El `checkpointLocation` garantiza la recuperación del estado en caso de fallo, permitiendo reanudar el pipeline desde el último microlote procesado sin pérdida ni duplicación de datos.

3.3. Almacenamiento estructurado (Base de Datos Relacional)

- PostgreSQL 15 (dockerizable): Almacenamiento de datos estructurados con modelo relacional:
 - Tabla `provincias_demograficas`: población, renta, tasa de paro, densidad
 - Tabla `administraciones_loteria`: ubicación, coordenadas GPS, provincia
 - Tabla `gasto_loteria_provincial`: recaudación trimestral por provincia
 - Tabla `sorteos_historicos_validados`: resultados con flag de validación cruzada
- Enriquecimiento: Spark Structured Streaming realiza joins en tiempo real con PostgreSQL para contextualizar eventos de apuestas con datos demográficos

3.3.1. Estrategia de ingesta incremental

La ingesta incremental se implementa de forma diferenciada según el tipo de fuente:

Fuentes relacionales (PostgreSQL). Se añade una columna `updated_at` `TIMESTAMP DEFAULT NOW()` en las tablas `sorteos_historicos_validados` y `administraciones_loteria`, actualizada automáticamente mediante un trigger de base de datos en cada `INSERT` o `UPDATE`. El pipeline consulta únicamente los registros donde `updated_at > $last_run`, siendo `$last_run` el valor almacenado en una tabla de control auxiliar:

```
CREATE TABLE pipeline_control (  
    pipeline_id    VARCHAR(50) PRIMARY KEY,  
    last_run       TIMESTAMP NOT NULL,  
    status         VARCHAR(20) -- 'success' | 'failed'  
);
```

Tras cada ejecución exitosa, el pipeline actualiza `last_run` con el timestamp de inicio del proceso, garantizando idempotencia en reruns.

Fuentes CSV periódicas (SELAE, INE). Al tratarse de ficheros de descarga completa con baja frecuencia de actualización (semestral y anual respectivamente), la incremental opera a nivel de fichero: se calcula el hash MD5 de cada descarga y se compara con el de la última versión procesada. Si ambos coinciden, el pipeline finaliza sin error y sin procesamiento redundante.

Capa Bronze como registro de auditoría. Independientemente de la fuente, cada microlote escrito en la capa Bronze de Delta incluye los metadatos `source`, `ingestion_timestamp` y `pipeline_run_id`, lo que permite trazabilidad completa y la ejecución segura de reruns sin generación de duplicados, apoyándose en la operación `MERGE` de Delta Lake.

De esta manera, tratamos la **Velocidad** porque se recurre al *streaming* para el último sorteo y a datos batch para históricos¹.

¹En la primera versión del documento, se presentaba el *streaming* simulado de manera que se entendía como un "batch planificado semanal", lo cual se refactorizó agregando un *message broker*, procesado no planificado, flujo de datos continuo y agregación constante (datos sintéticos)

4. Arquitectura Medallion y Delta Lake

Los tres pipelines de ingesta descritos anteriormente se integran en una Arquitectura Medallion de tres capas, cada una materializada como tablas Delta Lake. Esta estructura unifica el flujo batch y streaming en un modelo de calidad creciente.

4.1. Capa Bronze — datos en crudo

La capa Bronze recibe los datos de todas las fuentes sin aplicar ninguna transformación sobre el contenido original. Es *append-only* y actúa como registro de auditoría inmutable del dato de origen. Cada registro incorpora metadatos de trazabilidad:

```
CREATE TABLE bronze.sorteos_raw (  
  -- Contenido original preservado íntegramente  
  raw_content      STRING,  
  source           STRING, -- 'kaggle_es' | 'kaggle_uk' | 'selae'  
  ingestion_ts     TIMESTAMP,  
  pipeline_run_id  STRING  
) USING DELTA  
PARTITIONED BY (source, anio, mes);  
  
CREATE TABLE bronze.apuestas_raw (  
  -- Evento Kafka deserializado, sin transformar  
  raw_json         STRING,  
  kafka_offset     BIGINT,  
  kafka_partition  INT,  
  ingestion_ts     TIMESTAMP,  
  pipeline_run_id  STRING  
) USING DELTA  
PARTITIONED BY (DATE(ingestion_ts));
```

4.2. Capa Silver — datos validados y normalizados

La capa Silver aplica las transformaciones de calidad sobre Bronze: normalización de schemas, validación de rangos, deduplicación y enriquecimiento con datos demográficos. Es la **fente de verdad** para análisis y consumo por la capa Gold.

Los registros que no superan validación se derivan a tablas de cuarentena paralelas (`silver.sorteos_quarantine`, `silver.apuestas_quarantine`) sin interrumpir el pipeline.

```
CREATE TABLE silver.sorteos_validados (  
  fecha_sorteo     DATE,  
  dia_semana       STRING,  
  numero_1         INT, -- validado: rango 1-50  
  numero_2         INT,  
  numero_3         INT,  
  numero_4         INT,  
  numero_5         INT,  
  estrella_1       INT, -- validado: rango 1-12  
  estrella_2       INT,  
  recaudacion      DECIMAL(14,2),  
  pais_origen      STRING,  
  -- Metadatos de linaje  
  source_run_id    STRING,  
  validated_at     TIMESTAMP  
) USING DELTA  
PARTITIONED BY (anio, mes);
```

```

CREATE TABLE silver.apuestas_enriquecidas (
  timestamp          TIMESTAMP,
  id_transaccion     STRING,
  codigo_provincia   STRING,
  id_administracion  STRING,
  numeros            ARRAY<INT>,
  estrellas          ARRAY<INT>,
  importe            DECIMAL(8,2),
  tipo_apuesta       STRING,
  canal              STRING,
  -- Campos enriquecidos desde provincias_demograficas
  nombre_provincia   STRING,
  poblacion_total    INT,
  renta_media_hogar  DECIMAL(10,2),
  tasa_paro          FLOAT
) USING DELTA
PARTITIONED BY (DATE(timestamp));

```

El enriquecimiento se realiza mediante un join con `provincias_demograficas` leída como *static Data-Frame* en cada microlote del pipeline streaming. Cuando el pipeline batch actualiza esta tabla (tras la carga anual de INE), el pipeline streaming deberá ser relanzado para incorporar los nuevos datos, coordinación que recae sobre la orquestación Airflow.

4.3. Capa Gold — agregaciones para consumo

La capa Gold contiene las métricas precomputadas listas para consumo directo por dashboards o por el motor de probabilidad contextual. Se actualiza desde Silver tanto por el trigger streaming (cada 30 segundos) como por los pipelines batch tras cada carga.

```

CREATE TABLE gold.tendencias_numericas (
  periodo           DATE,
  numero            INT,
  frecuencia_absoluta INT,
  frecuencia_relativa FLOAT,
  racha_ausencia    INT, -- sorteos consecutivos sin aparecer
  updated_at        TIMESTAMP
) USING DELTA;

CREATE TABLE gold.agregaciones_provinciales (
  window_start      TIMESTAMP,
  window_end        TIMESTAMP,
  codigo_provincia   STRING,
  gasto_acumulado    DECIMAL(14,2),
  num_transacciones INT,
  canal_predominante STRING,
  updated_at        TIMESTAMP
) USING DELTA
PARTITIONED BY (DATE(window_start));

```

4.4. Programación temporal e integración batch-streaming

Los pipelines batch operan en cadencias diferenciadas que determinan la frecuencia de actualización de las capas Silver y Gold, y por tanto la coherencia del enriquecimiento realizado por el pipeline streaming.

Fuente	Cadencia	Tablas Delta afectadas	Impacto en streaming
INE	Anual (enero)	<code>silver.provincias_demograficas</code>	Alto — requiere relanzar Spark
SELAE puntos venta	Semestral	<code>silver.administraciones_loteria</code>	Bajo — solo análisis territorial
Kaggle + scraping SELAE	Bisemanal (ma/vi)	<code>silver.sorteos_validados</code>	Medio — recalibra índice Gold

Cuadro 2: Cadencias de ejecución de los pipelines batch y su impacto en el pipeline streaming.

Integración con el pipeline streaming. La coordinación entre batch y streaming se articula en dos puntos críticos:

- **Actualización de tablas de enriquecimiento:** cuando el pipeline batch de INE completa su carga y validación en Silver, el pipeline streaming debe ser relanzado para que el *static DataFrame* de `provincias_demograficas` incorpore los nuevos valores demográficos. Hasta ese relanzamiento, el streaming opera con los datos del período anterior.
- **Recálculo Gold post-sorteo:** el pipeline batch bisemanal recalcula las agregaciones de tendencias numéricas en Gold tras cada sorteo. Esta escritura debe coordinarse con el cierre de las ventanas streaming activas sobre las mismas particiones Delta, evitando conflictos de escritura concurrente. Delta Lake gestiona el aislamiento transaccional, pero la secuencia de ejecución debe estar orquestada explícitamente.

Orquestación. Ambos puntos de coordinación recaen sobre Apache Airflow, que gestiona las dependencias entre pipelines mediante sensores y mecanismos de comunicación entre DAGs. Esta integración se detalla en la sección de arquitectura de orquestación².

²La descripción de DAGs, sensores, XCom y mecanismos Push/Pull entre DAGs se aborda en el documento de arquitectura Airflow, complementario a esta memoria.

5. Datos necesarios

5.1. Datos históricos Euromillones (Batch)

Obtenidos de Kaggle (datasets español y británico) y scraping SELAE:

- Año
- Mes
- Día
- Fecha_sorteo
- Día_semana
- Número_1
- Número_2
- Número_3
- Número_4
- Número_5
- Estrella_1
- Estrella_2
- Recaudación_euros (cuando exista)
- Bote_acumulado
- País_origen (ES/UK, para trazabilidad)

5.2. Datos demográficos provinciales (Batch - INE)

Datos estructurados para enriquecimiento contextual:

- Código_provincia
- Nombre_provincia
- Comunidad_autónoma
- Población_total
- Densidad_población (hab/km²)
- Renta_media_hogar (€/año)
- Tasa_desempleo (%)
- PIB_per_capita (€)

5.3. Datos de puntos de venta (Batch - SELAE CSV)

Geolocalización de administraciones para análisis territorial:

- ID_administración (único SELAE)
- Nombre_comercial
- Dirección_completa
- Código_postal
- Municipio
- Código_provincia
- Latitud
- Longitud
- Tipo_establecimiento (administración/expendeduría/receptor)
- Fecha_última_actualización

5.4. Datos de streaming en tiempo real (Kafka)

Eventos sintéticos de transacciones de apuesta:

- Timestamp (ISO 8601)
- ID_transacción
- Código_provincia
- ID_administración
- Números_apostados (array de 5 números: 1-50)
- Estrellas_apostadas (array de 2 estrellas: 1-12)
- Importe (€)
- Tipo_apuesta (simple/múltiple/aleatorio)
- Canal (presencial/online/app_móvil)

5.5. Datos post-sorteo (Streaming - SELAE scraping)

Resultado oficial publicado tras cada sorteo (martes/viernes):

- Fecha_sorteo
- Número_1 ... Número_5
- Estrella_1, Estrella_2
- Bote_sorteo (€)
- Recaudación_españa (€)
- Timestamp_publicación

Esquema de un evento Kafka

```
{
  "timestamp": "2026-01-03T18:45:23.123Z",
  "id_transaccion": "TXN-20260103-184523-001",
  "codigo_provincia": "28",
  "id_administracion": "ADM-28-001",
  "tipo_apuesta": "simple|multiple|aleatorio",
  "numeros": [7, 23, 34, 41, 49],
  "estrellas": [3, 11],
  "importe": 2.50,
  "canal": "presencial|online|app"
}
```

Datos Estructurados (BD)

En este apartado se expondrán las tablas (creables para Postgres propuesta anteriormente) que conformarán la Base de Datos.

```
CREATE TABLE provincias_demograficas (
  codigo_provincia VARCHAR(2) PRIMARY KEY,
  nombre_provincia VARCHAR(100),
  poblacion_total INTEGER,
  densidad_poblacion FLOAT,
  renta_media_hogar DECIMAL(10,2),
  tasa_paro FLOAT,
  ccaa VARCHAR(50)
);

CREATE TABLE administraciones_loteria (
  id_administracion VARCHAR(20) PRIMARY KEY, -- ID oficial SELAE
  nombre_comercial VARCHAR(200),
  direccion VARCHAR(300),
  codigo_postal VARCHAR(5),
  municipio VARCHAR(100),
  codigo_provincia VARCHAR(2),
  latitud DECIMAL(10,8),
  longitud DECIMAL(11,8),
  tipo_establecimiento VARCHAR(50), -- 'Administración', 'Expendeduría', 'Receptor'
  fecha_actualizacion DATE,
  FOREIGN KEY (codigo_provincia) REFERENCES provincias_demograficas(codigo_provincia)
);

CREATE TABLE gasto_loteria_provincial (
  codigo_provincia VARCHAR(2),
  anio INTEGER,
  trimestre INTEGER,
  gasto_total_euromillones DECIMAL(12,2),
  num_boletos_vendidos INTEGER,
  gasto_per_capita DECIMAL(8,2),
  PRIMARY KEY (codigo_provincia, anio, trimestre),
  FOREIGN KEY (codigo_provincia) REFERENCES
    provincias_demograficas(codigo_provincia)
);
```

6. Arquitectura técnica y flujo de datos

6.1. Stack tecnológico

- **Ingesta batch:** Python + Pandas (descarga CSV Kaggle/INE)
- **Web scraping:** Selenium + BeautifulSoup (SELAE)
- **Base de datos:** PostgreSQL 15 (datos estructurados)
- **Message broker:** Apache Kafka + Zookeeper (streaming)
- **Procesamiento batch:** Apache Spark 3.5 (análisis históricos)
- **Procesamiento streaming:** Spark Structured Streaming (ventanas temporales)
- **Almacenamiento distribuido:** HDFS / MinIO (data lake)
- **Orquestación:** Apache Airflow (pipelines programados)
- **Visualización:** Apache Superset + Grafana
- **Contenedores:** Docker + Docker Compose

Nota: Todas las tecnologías son gratuitas, open-source y dockerizables en un sistema Linux (tipo Ubuntu).

6.2. Flujo de datos

6.2.1. Pipeline batch (ejecución diaria):

1. Airflow descarga CSVs de Kaggle (España + UK)
2. Validación cruzada: Spark compara números ganadores por fecha
3. Backfill: Completa sorteos faltantes desde dataset UK
4. ETL: Normaliza monedas (£ → €), carga en PostgreSQL
5. Descarga datos INE/Open Data, carga en PostgreSQL

6.2.2. Pipeline streaming (ejecución continua):

1. Generador sintético produce eventos de apuesta → Kafka (**apuestas-raw**)
2. Spark Streaming consume eventos, agrega en ventanas de 1 minuto
3. Join con PostgreSQL: enriquece con datos demográficos
4. Calcula métricas: gasto acumulado 48h, números más apostados, índice de probabilidad
5. Publica resultados → Kafka (**probabilidad-provincial**)
6. Grafana visualiza métricas en tiempo real

6.2.3. Post-sorteo (martes y viernes):

1. Scraper SELAE obtiene resultado oficial
2. Publica a Kafka (**resultados-sorteo**)
3. Spark actualiza PostgreSQL con nuevo sorteo
4. Recalcula métricas históricas (números calientes/fríos)